

CO3-AT1-Part1

Coding Challenge

#=====

Vanilla Policy Gradient & Reinforce

Name:N.Akshaya

Reg.no:192425129

Course code:MLA0305

Course name: REINFORCEMENT LEARNING

Slot:B

Python code:

```
# CODING CHALLENGE
```

```
# COMPARISON OF VANILLA POLICY GRADIENT AND REINFORCE
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import gymnasium as gym
```

```
# PARAMETERS
```

```
EPISODES = 500
```

```
LEARNING_RATE = 0.01
```

```
GAMMA = 0.99
```

```
# ENVIRONMENT
```

```
env = gym.make("CartPole-v1")
```

```
STATE_SIZE = env.observation_space.shape[0]
```

```
ACTION_SIZE = env.action_space.n
```

```
# SOFTMAX
```

```
def softmax(x):
```

```
    x = x - np.max(x)
```

```
    exp_x = np.exp(x)
```

```
    return exp_x / np.sum(exp_x)
```

```
# SELECT ACTION
```

```
def select_action(state, weights, bias):
```

```
    preferences = np.dot(state, weights) + bias
```

```
    probabilities = softmax(preferences)
```

```
    action = np.random.choice(
```

```
        ACTION_SIZE,
```

```
        p=probabilities
```

```
    )
```

```
    return action, probabilities
```

```
# DISCOUNTED RETURNS
```

```
def discounted_returns(rewards):
```

```
    returns = []
```

```
    G = 0
```

```
    for reward in reversed(rewards):
```

```
        G = reward + GAMMA * G
```

```
    returns.insert(0, G)
```

```

    return np.array(returns)

# VANILLA POLICY GRADIENT

def train_vpg():

    weights = np.zeros(

        (STATE_SIZE, ACTION_SIZE)

    )

    bias = np.zeros(ACTION_SIZE)

    rewards_history = []

    for episode in range(EPISODES):

        state, info = env.reset()

        states = []

        actions = []

        rewards = []

        probabilities_list = []

        done = False

        while not done:

            action, probabilities = select_action(

                state,

                weights,

                bias

            )

            next_state, reward, terminated, truncated, info = env.step(

                action

            )

```

```

states.append(state)

actions.append(action)

rewards.append(reward)

probabilities_list.append(probabilities)

state = next_state

done = terminated or truncated

# VPG: USE TOTAL EPISODE RETURN

episode_return = sum(rewards)

for state, action, probabilities in zip(

    states,

    actions,

    probabilities_list

):

    gradient = -probabilities.copy()

    gradient[action] += 1

    weights += (

        LEARNING_RATE

        * episode_return

        * np.outer(state, gradient)

    )

    bias += (

        LEARNING_RATE

        * episode_return

```

```

        * gradient
    )

    rewards_history.append(
        episode_return
    )

    return rewards_history

# REINFORCE

def train_reinforce():

    weights = np.zeros(
        (STATE_SIZE, ACTION_SIZE)
    )

    bias = np.zeros(ACTION_SIZE)

    rewards_history = []

    for episode in range(EPIISODES):

        state, info = env.reset()

        states = []

        actions = []

        rewards = []

        probabilities_list = []

        done = False

        while not done:

            action, probabilities = select_action(
                state,
                weights,

```

```

        bias
    )

    next_state, reward, terminated, truncated, info = env.step(
        action
    )

    states.append(state)

    actions.append(action)

    rewards.append(reward)

    probabilities_list.append(probabilities)

    state = next_state

    done = terminated or truncated

# REINFORCE: DISCOUNTED RETURNS
returns = discounted_returns(rewards)

# Normalize returns
returns = (
    returns - np.mean(returns)
) / (
    np.std(returns) + 1e-8
)

for state, action, G, probabilities in zip(
    states,
    actions,
    returns,
    probabilities_list

```

```

):

    gradient = -probabilities.copy()

    gradient[action] += 1

    weights += (

        LEARNING_RATE

        * G

        * np.outer(state, gradient)

    )

    bias += (

        LEARNING_RATE

        * G

        * gradient

    )

    rewards_history.append(

        sum(rewards)

    )

    return rewards_history

# TRAIN BOTH ALGORITHMS

print("=====")

print(" VANILLA POLICY GRADIENT vs REINFORCE")

print("=====")

print("\nTraining Vanilla Policy Gradient...")

vpg_rewards = train_vpg()

```

```

print("Vanilla Policy Gradient completed.")

print("\nTraining REINFORCE...")

reinforce_rewards = train_reinforce()

print("REINFORCE completed.")

# SUMMARY

vpg_initial = np.mean(
    vpg_rewards[:50]
)

vpg_final = np.mean(
    vpg_rewards[-50:]
)

vpg_max = max(vpg_rewards)

reinforce_initial = np.mean(
    reinforce_rewards[:50]
)

reinforce_final = np.mean(
    reinforce_rewards[-50:]
)

reinforce_max = max(
    reinforce_rewards
)

print("\n=====")

print("      COMPARISON SUMMARY")

print("=====")

```



```

print("\nVanilla Policy Gradient")

print("-----")

print("Initial Average Reward :", round(vpg_initial, 2))

print("Final Average Reward  :", round(vpg_final, 2))

print("Maximum Reward      :", round(vpg_max, 2))

print("\nREINFORCE")

print("-----")

print(
    "Initial Average Reward :",
    round(reinforce_initial, 2)
)

print(
    "Final Average Reward  :",
    round(reinforce_final, 2)
)

print(
    "Maximum Reward      :",
    round(reinforce_max, 2)
)

# DETERMINE BETTER ALGORITHM

print("\n=====")

print("      FINAL RESULT")

print("=====")

```

```
if vpg_final > reinforce_final:

    print(

        "Better Algorithm : Vanilla Policy Gradient"

    )

elif reinforce_final > vpg_final:

    print(

        "Better Algorithm : REINFORCE"

    )

else:

    print(

        "Result : Both algorithms have similar performance"

    )

# GRAPH 1

# TRAINING PERFORMANCE COMPARISON

plt.figure(figsize=(11, 6))

plt.plot(

    vpg_rewards,

    color="blue",

    alpha=0.35,

    linewidth=1,

    label="Vanilla Policy Gradient"

)
```

```
plt.plot(
    reinforce_rewards,
    color="red",
    alpha=0.35,
    linewidth=1,
    label="REINFORCE"
)

# Moving averages
window = 20

vpg_average = np.convolve(
    vpg_rewards,
    np.ones(window) / window,
    mode="valid"
)

reinforce_average = np.convolve(
    reinforce_rewards,
    np.ones(window) / window,
    mode="valid"
)

plt.plot(
    range(window - 1, EPISODES),
    vpg_average,
    color="blue",
    linewidth=3,
```

```
    label="VPG 20-Episode Average"
)

plt.plot(
    range(window - 1, EPISODES),
    reinforce_average,
    color="red",
    linewidth=3,
    label="REINFORCE 20-Episode Average"
)

plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title(
    "Vanilla Policy Gradient vs REINFORCE"
)

plt.legend()

plt.grid(
    True,
    alpha=0.3
)

plt.tight_layout()

plt.show()

# GRAPH 2

# FINAL AVERAGE REWARD COMPARISON

plt.figure(figsize=(8, 5))
```

```
algorithms = [  
    "Vanilla Policy\nGradient",  
    "REINFORCE"  
]  
  
final_rewards = [  
    vpg_final,  
    reinforce_final  
]  
  
bars = plt.bar(  
    algorithms,  
    final_rewards,  
    color=["blue", "red"],  
    edgecolor="black"  
)  
  
plt.ylabel(  
    "Average Reward"  
)  
  
plt.title(  
    "Final 50-Episode Average Reward"  
)  
  
plt.grid(  
    axis="y",  
    alpha=0.3
```

```

)

# Display values on bars

for bar in bars:

    height = bar.get_height()

    plt.text(

        bar.get_x() + bar.get_width() / 2,

        height,

        f"{height:.2f}",

        ha="center",

        va="bottom",

        fontweight="bold"

    )

plt.tight_layout()

plt.show()

# GRAPH 3

# INITIAL VS FINAL PERFORMANCE

x = np.arange(2)

width = 0.35

plt.figure(figsize=(9, 5))

plt.bar(

    x - width / 2,

    [vpg_initial, reinforce_initial],

    width,

    color="green",

```

```
    edgecolor="black",  
    label="First 50 Episodes"  
)  
plt.bar(  
    x + width / 2,  
    [vpg_final, reinforce_final],  
    width,  
    color="orange",  
    edgecolor="black",  
    label="Last 50 Episodes"  
)  
plt.xticks(  
    x,  
    [  
        "Vanilla Policy Gradient",  
        "REINFORCE"  
    ]  
)  
plt.ylabel(  
    "Average Reward"  
)  
plt.title(  
    "Initial vs Final Performance"  
)
```

```
plt.legend()

plt.grid(

    axis="y",

    alpha=0.3

)

plt.tight_layout()

plt.show()

# GRAPH 4

# MAXIMUM REWARD COMPARISON

plt.figure(figsize=(8, 5))

plt.bar(

    ["Vanilla Policy Gradient", "REINFORCE"],

    [vpg_max, reinforce_max],

    color=["purple", "cyan"],

    edgecolor="black"

)

plt.ylabel(

    "Maximum Reward"

)

plt.title(

    "Maximum Reward Comparison"

)

plt.grid(
```



```

    axis="y",

    alpha=0.3

)

plt.tight_layout()

plt.show()

# CLOSE ENVIRONMENT

env.close()

print("\n=====")

print(" Experiment Completed Successfully")

print("=====")

```

Summary Result:

```

=====
VANILLA POLICY GRADIENT vs REINFORCE
=====

Training Vanilla Policy Gradient...
Vanilla Policy Gradient completed.

Training REINFORCE...
REINFORCE completed.

=====
                        COMPARISON SUMMARY
=====

Vanilla Policy Gradient
-----
Initial Average Reward : 10.44
Final Average Reward   : 9.22
Maximum Reward         : 60.0

REINFORCE
-----
Initial Average Reward : 28.04
Final Average Reward   : 249.94
Maximum Reward         : 500.0

=====
                        FINAL RESULT
=====
Better Algorithm : REINFORCE

```

Visualization Result:



